

(Sinh viên không được sử dụng tài liệu, Làm bài trên đề)

HỌ VÀ TÊN SV: MSSV: STT: PHÒNG THI:.....	<u>CÁN BỘ COI THI</u>
---	------------------------------

Đề gồm 2 phần: Câu trả lời ngắn và Câu hỏi tự luận. Tổng số câu hỏi: 10 câu.

PHẦN 1: CÂU TRẢ LỜI NGẮN

Câu 1 (1.0 điểm) (CLO1, CLO2)

Cho mảng một chiều a gồm n phần tử số nguyên, các phần tử có thứ tự tăng dần và được áp dụng giải thuật tìm kiếm tuyến tính để tìm kiếm phần tử có giá trị x trong mảng a , hãy thực hiện các yêu cầu sau:

- (0.5đ) Viết thuật toán tìm kiếm tuyến tính **có sử dụng điều kiện các phần tử tăng dần** của mảng theo giả thiết đã cho. Lưu ý: Nếu học viên viết thuật toán tuyến tính khác với yêu cầu thì chỉ tính 0.25 điểm.
- (0.5đ) Dựa vào thuật toán tuyến tính đã viết, hãy tính số lần so sánh $a[i] == x$ (với i là chỉ số phần tử đang xét của mảng a) trong các trường hợp tìm kiếm giá trị x nằm ở đầu mảng, cuối mảng và giữa mảng (chỉ nêu kết quả, không cần giải thích).

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Câu 2 (1.5 điểm) (CLO1)

Cho dãy 12 giá trị như sau: **40, 50, 4, 30, 45, 60, 20, 10, 15, 3, 90, 70**. Yêu cầu:

- a. (0.5đ) Vẽ cây nhị phân tìm kiếm T1 sau khi thêm lần lượt các giá trị ở dãy trên vào theo thứ tự từ trái sang phải. Lưu ý cây T1 rỗng trước khi thêm các giá trị trên.

- b. (0.5đ) Cho biết kết quả duyệt cây T1 theo thứ tự Right Left Node (RLN).

.....

- c. (0.5đ) Hãy vẽ lại cây T1 sau khi xóa lần lượt 60 và 4 sao cho T1 vẫn là cây nhị phân tìm kiếm. Nút thay thế được chọn (nếu có) là giá trị nhỏ nhất nhưng lớn hơn giá trị cần xóa.

Câu 3 (1.0 điểm) (CLO1, CLO2)

- a. (0.5đ) Điền vào chỗ trống để hoàn thành cài đặt của **thuật toán sắp xếp tăng dần** theo phương pháp Chọn trực tiếp sau: (Lưu ý: Mỗi vị trí trống chỉ điền một lệnh phù hợp)

```

void SelectionSort(int *a, const int &n) {
    for (int i = 0; i < n-1; i++) {
        int min = i;
        for (int j = .....; j < n; j++)
            .....
        swap(a[i], a[min]);
    }
}

```

- b. (0.5đ) Cho mảng *arr* gồm 10 phần tử sau: 1, 2, 3, 4, 5, 6, 8, 8, 8, 8. Hãy cho biết cần gọi bao nhiêu thao tác đổi chỗ **swap** trong hàm đã hoàn thành ở câu a để thực hiện sắp xếp mảng *arr* tăng dần?

Kết quả:

Câu 4 (1.0 điểm) (CLO1, CLO2)

Hãy điền vào vị trí trống bên dưới để hoàn chỉnh hàm **add_Y_after_X** thêm phần tử Y sau phần tử X trong danh sách liên kết đơn:

```

struct NODE {
    int info;
    NODE* pNext;
};
struct LIST {
    NODE* pHead;
    NODE* pTail;
};
void add_Y_after_X(LIST &L, int Y, int X){
    NODE *Q=searchNode(L, X);
    // hàm searchNode trả về node chứa X nếu tìm thấy X
    // trả về NULL nếu không tìm thấy X

    if (Q==NULL) .....;

    NODE* p=CreateNode(Y);

    p->pNext = .....;

    Q->pNext = .....;

    if(L.pTail==Q)

        L.pTail = .....;
}

```

Câu 5 (1.0 điểm) (CLO1, CLO2)

Cho biết cây B-Tree bậc 3 là một cây thỏa mãn các tính chất sau:

- Các khóa trong mỗi node được sắp xếp theo thứ tự tăng dần
- Tất cả node lá nằm trên cùng một mức
- Tất cả các node, trừ node lá, có **tối thiểu** 2 node con và **tối đa** 3 node con
- Nếu cây không rỗng, tất cả các node có **từ 1 cho đến 2 khóa (keys)**
- Một node không phải lá và có n khóa thì phải có n+1 node con

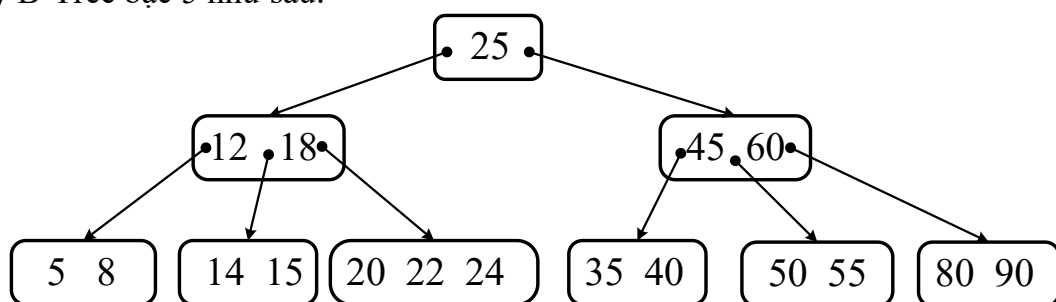
Vẽ cây B-Tree bậc 3 sau khi thêm lần lượt 15 khóa sau theo thứ tự từ trái sang phải: {10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80}. (Lưu ý cây B-Tree bậc 3 ban đầu rỗng)

Câu 6 (1.0 điểm) (CLO1, CLO2)

Cho biết cây B-Tree bậc 5 là một cây thỏa mãn các tính chất sau:

- Các khóa trong mỗi node được sắp xếp theo thứ tự tăng dần
- Tất cả node lá nằm trên cùng một mức
- Một node không phải lá và có n khóa thì phải có $n+1$ node con
- Nếu cây không rỗng, **node gốc có tối thiểu 1 khóa và tối đa 4 khóa**
- **Các node (khác gốc) có tối thiểu 2 khóa và tối đa 4 khóa**

Cho cây B-Tree bậc 5 như sau:



Hình 1. Cây B-Tree bậc 5

Lưu ý: - **Hai câu a và b làm độc lập với nhau**

- Quy ước:

- + Khi khóa cần xóa (gọi là x) không nằm ở node lá, ưu tiên chọn khóa thế mạng là khóa có giá trị lớn nhất mà nhỏ hơn x .
- + Khi có 02 lựa chọn node liền kề để thực hiện Catenation, ưu tiên chọn Catenate giữa node bị thiếu khóa với node liền trước.

a. (0.5đ) Vẽ lại cây B-Tree ở **Hình 1** sau khi xóa 20.

- b. (0.5đ) Vẽ lại cây B-Tree ở **Hình 1** sau khi xóa 60. (Nhắc: Xóa cây ở Hình 1, Không xóa cây kết quả ở câu a)

Câu 7 (1.0 điểm) (CLO1, CLO2)

Cho bảng băm với $M=11$ (TableSize=11), **Load factor là 0.6** và hai hàm băm sau:

$$h_1(\text{key}) = \text{key} \bmod 7$$

$$h_2(\text{key}) = 7 - (\text{key} \bmod 7)$$

Chèn các khóa {1, 8, 15, 22, 10, 31, 29, 36} theo thứ tự từ trái sang phải vào Bảng băm HT sử dụng phương pháp băm Double-Hashing với h_1 là hàm băm chính và h_2 là hàm băm phụ:

$$h(\text{key}, i) = (h_1(\text{key}) + i * h_2(\text{key})) \bmod \text{TableSize}$$

- a. (0.5đ) Hãy trình bày từng bước việc thêm các khóa vào Bảng băm HT bằng cách điền kết quả tính toán vào Bảng 1 bên dưới. (SV điền trực tiếp vào bảng)

key	$h_1(\text{key})$	$h_2(\text{key})$	Kết quả băm lại lần 1	Kết quả băm lại lần 2	Kết quả băm lại lần 3	Kết quả băm lại lần 4

Bảng 1: Minh họa từng bước tính toán trong quá trình thêm các khóa vào Bảng băm

b. (0.5đ) Điền các khóa được tính toán trên câu a vào Bảng băm HT ở Bảng 2.

Chỉ số	0	1	2	3	4	5	6	7	8	9	10
Khóa											

Bảng 2: Bảng băm HT sau khi thêm các khóa

Câu 8 (1.0 điểm) (CLO1)

a. (0.5đ) Cho bảng băm với TableSize=10 (chỉ số bảng băm từ 0 đến 9), hàm băm $h(key) = key \bmod 10$ và phương pháp băm lại Linear probing. Sau khi chèn và xóa một số phần tử trên bảng băm, ta được bảng sau:

Chỉ số	0	1	2	3	4	5	6	7	8	9
Khóa	50	41	DEL	22		55	DEL		DEL	28

Điền các khóa/ trạng thái tương ứng vào Bảng bên dưới sau khi Chèn thêm khóa 10 và 23 vào bảng băm đã cho ở trên. (SV điền trực tiếp vào bảng)

Chỉ số	0	1	2	3	4	5	6	7	8	9
Khóa										

Lưu ý: Các vùng nhớ trong bảng băm sẽ tồn tại 1 trong 3 trạng thái: Deleted (**ghi tắt: DEL**), Empty và Occupied. Khi minh họa bảng băm theo yêu cầu, đề nghị Học viên ghi rõ trạng thái tương ứng của các vùng nhớ trong Bảng băm, trong đó:

- Trạng thái **Empty** có thể để trống
- Trạng thái **Occupied** không phải ghi vì đã có giá trị cụ thể trong vùng nhớ tương ứng
- Trạng thái **Deleted (DEL)** phải được ghi rõ trong minh họa bảng băm.

b. (0.5đ) Điền vào chỗ trống để hoàn thành quy tắc vận hành của bảng băm sử dụng phương pháp xử lý đụng độ bằng băm lại trong **quá trình xóa** một giá trị x :

- Nếu băm giá trị x cần xóa vào vị trí có trạng thái **Empty** thì ta sẽ thực hiện:

.....

- Nếu băm giá trị x cần xóa vào vị trí có trạng thái **Deleted** thì ta sẽ thực hiện:

.....

- Nếu băm giá trị x cần xóa vào vị trí có trạng thái **Occupied** thì ta sẽ thực hiện:

.....

PHẦN 2: CÂU HỎI TỰ LUẬN

Câu 9 (1.0 điểm) (CLO2, CLO3)

Để lưu trữ một đa thức thừa một biến trong máy tính, ta có thể sử dụng cấu trúc **Danh sách liên kết đơn**. Trong đó, mỗi nút (Node) đại diện cho một đơn thức bao gồm hai thành phần: **hệ số** và **số mũ**.

Ví dụ đa thức $P(x) = 100x^{70} + 25x^{15} - 6x^3 + 9$ sẽ được lưu trữ dưới dạng một **danh sách các Node đơn thức** có số mũ giảm dần.

Một số cấu trúc dữ liệu gợi ý:

```
struct DONTHUC {
    float HeSo;
    int SoMu;
```

```
};

struct NODE {
    DATHUC info;
    NODE* pNext;
};
```

Cho trước các hàm hỗ trợ sau đã được cài đặt hoàn chỉnh (học viên không cần viết lại):

```
NODE* CreateNode(DATHUC x)
void AddTail(DATHUC &L, NODE* p)
void CreateEmptyDATHUC(DATHUC &poly)
DATHUC Input(float heso, int somu)
void Output(const DATHUC &poly)
```

Dựa vào hàm main dưới đây:

```
int main() {
    DATHUC poly1, poly2;
    Input(poly1);
    Input(poly2);

    cout << "sum: ";
    Output(Sum(poly1, poly2));

    return 0;
}
```

Học viên hãy thực hiện các công việc sau:

- (0.25đ) Định nghĩa cấu trúc dữ liệu DATHUC phù hợp để quản lý danh sách liên kết đơn lưu trữ một Đa thức.
- (0.25đ) Cài đặt hàm void Input(DATHUC &poly) để nhập đa thức từ bàn phím
 - Đầu tiên, cho phép người dùng nhập vào số lượng đơn thức.
 - Tiếp theo, lần lượt nhập hệ số và số mũ cho từng đơn thức.
 - **Quy ước:** Các đơn thức được nhập vào theo thứ tự **số mũ giảm dần**.
- (0.5đ) Cài đặt hàm DATHUC Sum(DATHUC poly1, DATHUC poly2) để cộng hai đa thức, kết quả trả về là một đa thức mới.
 - **Lưu ý:** Đa thức kết quả cũng phải đảm bảo thứ tự số mũ giảm dần và không chứa các đơn thức có hệ số bằng 0.

.....

.....

.....

.....

.....

[illegible]

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Câu 10 (0.5 điểm) (CLO2, CLO3)

Viết hàm `int EvenMax(TNODE *Root)` nhận đầu vào `Root` là con trỏ đến gốc của cây nhị phân tìm kiếm, trả lại giá trị key chẵn lớn nhất trong các nút của cây nhị phân tìm kiếm đã cho.

```
struct TNODE {
    int key;
    TNODE* pLeft;
    TNODE* pRight;
};
typedef TNODE* TREE;
```

.....

.....

.....

